

Labo: REST-webservices

Inhoudstafel

Context	2
Voorbeeld	3
Opgaven	4
1 Ophalingen ophijsten	4
2 Details van een klant opvragen	4
3 Details van een klant wijzigen	5
4 Een klant verwijderen	5

Context



RecycleHub

RecycleHub is een fictief recyclagebedrijf dat afval ophaalt bij particulieren in Gent. Het bedrijf gebruikt al jaren een intern back-endsysteem dat onder andere instaat voor het wegen en aanrekenen van afval. Containers van klanten bevatten elk een chip die gekoppeld is aan hun account. Bij een ophaling leest de boordcomputer van een vuilniswagen automatisch de chip uit en registreert deze het gewicht van het afval. Bij aankomst op de verwerkingssite wordt deze informatie rechtstreeks in de centrale back-end gevoed.

De helpdesk van RecycleHub krijgt geregeld telefoontjes van klanten die hun gegevens wensen aan te passen (omwille van een verhuis, een verkeerd saldo, ...). Op dit moment kan dit enkel met tussenkomst van de IT-afdeling, die de gegevens rechtstreeks in de databank moet wijzigen. Om dit proces efficiënter te laten verlopen, wenst RecycleHub een webapp waarmee helpdeskmedewerkers deze aanpassingen zelf kunnen uitvoeren. Jij kreeg de opdracht om deze te ontwikkelen.

Je collega's hebben alvast een REST-webservice gemaakt, zodat jij vlot via HTML-requests met de back-end kan interageren. De API biedt drie centrale resources: customers, bins en pickups. De verschillende endpoints worden beschreven in de documentatie:

documenter.getpostman.com/view/54706190/2sBXwjutTc

GET customers

GET customers/{id}

PUT customers/{id}

DEL customers/{id}

POST customers

GET bins

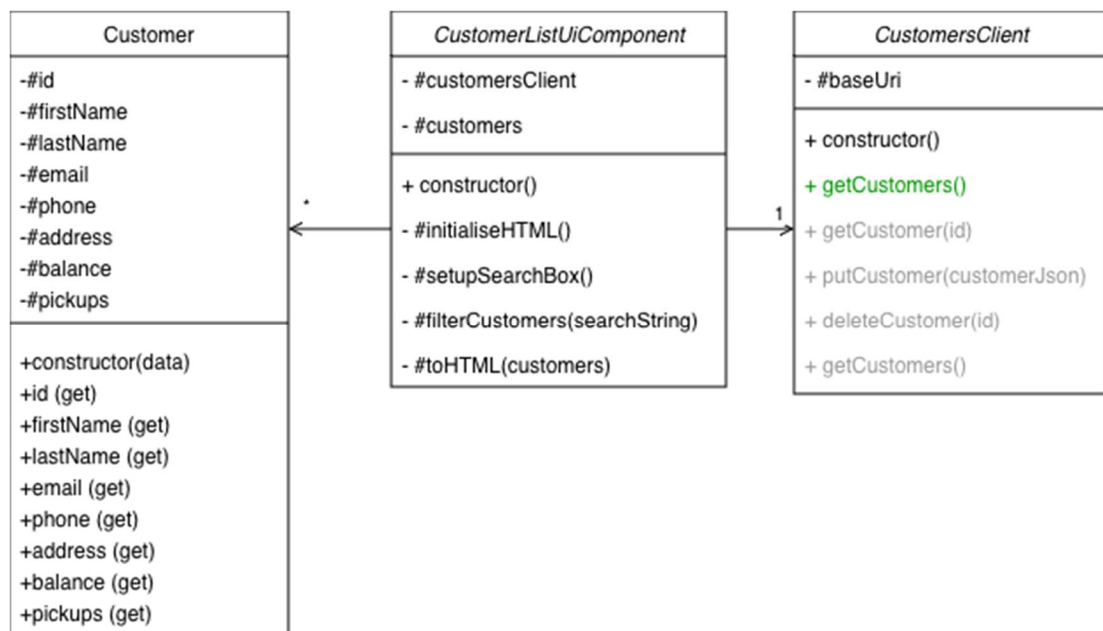
GET pickups

Voorbeeld

Open `index.html` met Live Server en navigeer naar "Klanten". Je krijgt een lijst van bootstrap cards te zien, opgevuld met de klanten verkregen via een **GET**-request voor customers aan de REST-webservice. De client-side logica zit in dit geval verspreid over drie JS-files:



Inspecteer de code in bovenvermelde componenten en zorg dat je goed begrijpt hoe de flow ertussen verloopt. Onderstaand klassendiagram kan je hierbij helpen.



Opgaven

1 Ophalingen ophalingen

Navigeer naar de pagina met "Ophalingen". Vul naar analogie met `customer_list.html`, de content van `pickup_list.html` op met alle ophalingen die de API je biedt via de `pickups` resource.

Ophalingen

2026-06-10
PMD

2026-06-15
GFT

2026-06-18
GI AS



Zoek in de API-docs welke endpoint je moet aanspreken en op welke manier. Roep die endpoints vervolgens op in de `getPickups`-methode van `pickup_client.js`.

Vorm de JSON-response die je terugkrijgt om naar een `Pickup`-object, waarvan je de klasse implementeert in `pickup.js`.

Vorm elk `Pickup`-object vervolgens om naar een bootstrap card (zie `customer_list.html`) in de `#toHTML`-methode van `pickup_list_ui_component.js`.

2 Details van een klant opvragen

Navigeer opnieuw naar de webpagina met "Klanten". Wanneer je op de knop "Details" van een klant klikt, wordt je doorverwezen naar `customer_details.html`. Merk op dat de `id` van de klant voorkomt als parameter in de url (`?id=...`). Populeer de velden van het formulier met de data van de overeenkomstige customer.



Volg opnieuw de keten van JS-files waar de `<script>`-tag onderaan in `customer_details.html` in naar verwijst. Zo kom je tot de bijhorende methode van de `CustomerClient`. Implementeer deze.

3 Details van een klant wijzigen

Wanneer de knop “Wijzigingen opslaan” wordt aangeklikt, wordt er opnieuw een ketting van functie-oproepen in gang gezet. Ditmaal leidt dit tot de `putCustomer`-methode van de `CustomerClient`.

Implementeer hier een PUT-request naar de juiste endpoint.



Zoek opnieuw in de API-docs op welke manier je deze endpoint moet aanspreken. Ditmaal zal je niet enkel een URI moeten meegeven aan `fetch`.

Je HTTP-request moet nu een body krijgen die beschrijft hoe de geüpdatete customer eruit moet zien. Dit onder de vorm van JSON. Je zal de data uit het formulier moeten omvormen naar dit JSON-formaat. Doe dit in de `#toCustomerJson`-methode van de `CustomerDetailsUiComponent`.



Aangezien de webservice slechts een mock is, zit er geen “echte” back-end achter de API. De data van de customer wordt dus niet echt wordt geüpdatet. Indien je de HTML-request juist maakt, zal de response wel correct nabootsen wat de response van een echte back-end zou zijn.

4 Een klant verwijderen

Implementeer nu ook de `deleteCustomer`-methode van `CustomerClient` om de knop “Verwijder klant” in werking te laten gaan.

Details van de klant

Voornaam	Achternaam	
Arno	Temmerman	
E-mail		
arno.temmerman@email.com		
Telefoon		
+32 471 22 33 44		
Straat	Postcode	Gemeente
Stationsstraat 14	9000	Gent
Saldo		
26.3		
Wijzigingen opslaan		
Verwijder klant		